So, summing up, `_heapify` keeps improving even at larger branching factors, although we can also say it practically plateaus after `D==13`, but this behavior is not caused by methods `top` and `_push_down`, which do behave as expected.

There could be a few explanations for how `_heapify` running time grows:

1. It's possible that, checking larger branching factors, we discover that there is a minimum (we just haven't found it yet).
2. The performance of this method heavily depends on the order of insertions: with sorted data it performs way better than random data.
3. Implementation-specific details makes the contribution of calls to `_push_down` less relevant over the total running time.

But... are we sure that this is in contrast with theory?

You should know by now that I like rhetorical questions: this means it is time to get into some Math.

And indeed, if we take a closer look at section 2.6.7, we can find out that the number of swaps performed by `_heapify` is limited by:

$$\frac{n}{D} \cdot \sum_{h=0}^{\lfloor log_D(n) \rfloor} \left\lceil \frac{h}{D^h} \right\rceil$$